Assignment - 03

Ankit Kumar
22BCON1068

Subject: Adv. Java

Date: 10/10/24

- Section - A (2x5=10)

1. What is the role of Java.rmi.Naming class?

Ans The Java.rmi.Naming class provides methods for locating, binding or unbinding remote objects in the RMI (Remote method Invocation) registry. it allows you to register a remote object with a name, look up remote object by name, and remove binding for remote objects.

2. What is difference between bind() and rebind() methods of Naming class?

Ans bind(): Binds a remote object to a name, but throws an exception if the name is already bound.

rebind(): Binds a remote object to a name, replacing any existing binding for that name without throwing an exception.

2. What is the Role of Remote interface in RMI?

Ans The Remote interface in RMI serves as a marker interface that identifies an object as capable of being invoked remotely. Any object that implements this interface can have its methods called from a remote client across the network. it ensures that the object is treated as a remote object by the RMI system.

4. Define Marshalling and Demarshalling in RMI?

Ans - Marshalling - The process of converting method parameters or return values into a format that can be sent over the network.

- Demarshalling - The reverse process, where the received byte system is converted back into the original parameters or return value on the remote end.

Oct 14, 2024, 18:02

22BCON1068
Ankit kumar

Subject:

Date: / / 24

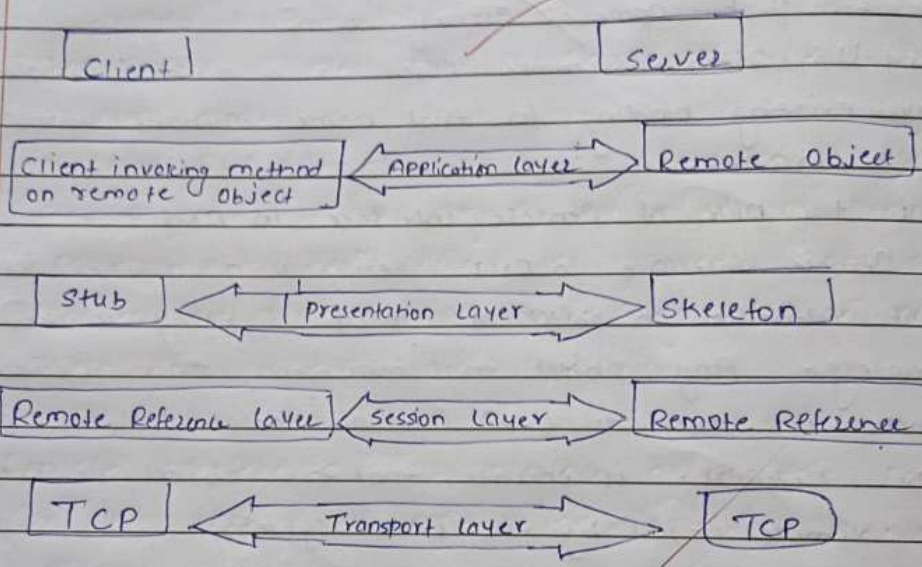
5. What is the purpose of using RMI security Manager in RMI?

Ans The purpose of the RMI security manager is to enforce security policies when downloading and executing remote code. It ensures that the application adheres to security restrictions, such as file access, socket communication, or execution of potentially untrusted code in RMI-based applications.

- Section-B (9×9=21)

1. What are the layers of RMI architecture?

Ans



The RMI architecture consists of three main layers:

(i) Application layer:- Contain the client and server programs that define the remote interfaces and implement remote objects.

(ii) Stub and Skeleton layer:- Acts as an intermediary between the application layer and the transport layer. Stubs on the client side

Oct 14, 2024, 18:02

22BCON1068
Ankit Kumar

Subject : _____

Date : __/__/____

and skeletons on the server side handle method invocation and response.

(iii) Transport Layer :- Handles the communication between the client and server, managing connection setup, data transfer, and remote object references over the network. it typically uses TCP/IP.

2. What is the function of Java.net.UnknownHostException?

Ans The Java.net.UnknownHostException is not actually a function, it's a subclass of IOException in Java, and it is thrown when an attempt to resolve a hostname to an IP address fails. This is typically when:

(i) The Hostname is incorrect or does not exist. for example, if you are trying to access a server by a domain name that is not registered or not reachable, Java will throw an UnknownHostException.

(ii) The DNS server cannot resolve the hostname. This might happen due to network configuration issues, DNS server problems, or if the DNS server is unreachable.

ex :-

```
import Java.net.InetAddress;
import Java.net.UnknownHostException;

public class host {
    public static void main(String[] args) {
        try {
            InetAddress address = InetAddress.getByName("non-
                existentwebsite.com");
        }
```

```
catch (UnknownHostException e) {
    System.out.println("UnknownHostException : " + e.getMessage());
}
}
```

O/P: UnknownHostException: nonexistentwebsite.com

in this example, trying to resolve the hostname "nonexistentwebsite.com" will throw an UnknownHostException because the domain does not exist.

2. What are the different types of classes in RMI. Write the snippet also.

Ans in Java RMI (Remote Method Invocation), several types of classes and interfaces play a role in enabling remote communication bet^n client and server applications. These classes fall into a few broad categories based on their role. Here is the main types :-

(i) Remote Interface :- This interface defines the methods that can be invoked remotely. It extends the Java.rmi.Remote Interface. Each method must throw a RemoteException.

ex.

```
import Java.rmi.Remote;
import Java.rmi.RemoteException;
public interface Calculator extends Remote {
    Public int add(int a, int b) throws RemoteException;
    public int subtract(int a, int b) throws RemoteException;
}
```

Subject : _____ Date : __/__/____

(ii) Remote object implementation :- This class implements the remote interface and provides the actual functionality of the methods defined in the remote interface. it extends "Java.rmi.server.UnicastRemoteObject".

ex.

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
public class Calculator extends UnicastRemoteObject implements Calculator {
    protected Calculator() throws RemoteException {
        super();
    }
    public int add(int a, int b) throws RemoteException {
        return a+b;
    }
    public int subtract(int a, int b) throws RemoteException {
        return a-b;
    }
}
```

(iii) Server class (RMI server) :- This class is responsible for creating the instance of the remote object and binding it to the RMI registry so that clients can look it up.

ex

```
import java.rmi.registry.LocalRegistry;
import java.rmi.registry.Registry;
Public class server {
public static void main(String[] args) {
try {
Calculator calculator = new Calculator1();
Registry reg = LocateRegistry.CreateRegistry(1099);
reg.bind("Calculatorservice", calculator);
S.O.Pln("Calculator service is ready.");
}
```

Oct 14, 2024, 18:02

Subject :
Date : __/__/____

```
        catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

(iv) Client class (RNI client) :- The client looks up the remote object in the RMI registry and invokes methods on it as if it were a local object.

ex:

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
public class Client {
    public static void main(String[] args) {
        try {
            Registry reg = LocateRegistry.getRegistry("localhost", 1099);
            Calculator calculator = (Calculator) registry.lookup("CalculatorService");
            S.O.P("Addition:" + calculator.add(5,3));
            S.O.P("subtraction:" + Calculator.subtract(9,4));
        }
        catch (Exception e) {
            System.out.println(e)
        }
    }
}
```

(v) RMI Registry :- The RMI registry is a name service that allows clients to look up remote objects. It typically runs on port 1099 (default port) and is used to register and lookup remote objects by name.

22BCON1068
Ankit kumar

Subject : _____ Date : __/__/____

- Section - C (11 * 3 = 33)

1. Explain the serialization and deserialization. ?

Ans Serialization :- Serialization is the process of converting an object into a byte stream so that it can be :-
- save to a file or a database
- Sent over a network to another JVM or another system.
- Stored in memory for later use.

The main purpose is to persist the state of an object or transfer it to another environment.

How Serialization works :-
- in Java, an object is serialized using the "ObjectOutputStream" class, and the object must implement the "Serializable" interface.
- The fields of the object are written to the byte stream in such a way that the state of the object can be reconstructed later.

ex :-

```
import Java.io.FileOutStream;
import Java.io.ObjectOutputStream;
import Java.io.Serializable;
class Employee implements Serializable {
    String name;
    int id;

    Public Employee (String name, int id) {
        this.name = name;
        this.id = id;
    }
}

public class Ex {
    public static void main(String[] args) {
```

Oct 14, 2024, 18:02

Ankit Kumar
22BCON1068

Subject :

Date : __/__/____

```
try {
Employee emp = new Employee("Ankit Rai", 1068);
FileOutputStream fileOut = new FileOutputStream("employee.txt");
ObjectOutputStream out = new ObjectOutputStream(fileOut);

out.writeObject(emp);
out.close();
fileOut.close();
System.out.println("Employee object serialized.");
} catch (Exception e) {
System.out.println(e);
}
}
}
```

- Deserialization :- Deserialization is the reverse process of serialization. it is the process of converting the byte system back into a copy of the original object.

How Deserialization Works :-

- in Java, an object is deserilized using the "ObjectInputStream" class.
- The object that was serialized earlier is reconstruted from the byte system.

Ex:

```
import java.io.FileInputStream;
import java.io.ObjectInputStream;
public class DeserEx {
public static void main(String[] args) {
try {
FileInputStream fileIn = new FileInputStream("employee.txt");
ObjectInputstream in = new ObjectInputstream(fileIn);
```

Oct 14, 2024, 18:02

22BCON1068
Ankit Kumar

Subject : Date : __/__/____

```
Employee emp = (Employee) in.readObject();
in.close();
fileIn.close();
System.out.println("Employee Name: " + emp.name);
System.out.println("Employee ID: " + emp.id);
} catch (Exception e)
{
System.out.println();
}
}
}
```

2. What is the method that is used by RMI client to connect to remote RMI server. Implement the RMI Application Program.

Ans In Java RMI (Remote Method Invocation), the RMI client uses the "lookup()" method of the "Java.rmi.registry.Registry" class to connect to a remote RMI server. The "lookup()" method retrives a reference to the remote object from the RMI registry using a name associated with that object.

Steps to Implement an RMI application.

(i) Define a Remote Interface :- Define the methods that the server will expose and the client will call.

(ii) Implement the Remote Object :- Implements the methods define in the remote interface.

(iii) Create and Start the Server :- The server creates an instance of the remote object and binds it to the RMI registry.

(iv) Create the client :- The client looks up the remote object and invokes the methods remotely.

22BCON1068
Ankit Kumar

Subject : Date : __/__/__

• Complete RMI Application program.

(i) Remote Interface :-

```
import java.rmi.*;
Public interface Adder extends Remote {
  Public int add(int x, int y) throws RemoteException;
}
```

(ii) Remote object implementation :-

```
import Java.rmi.*;
import Java.rmi.server.*;
Public class AdderRemote extends UnicastRemoteObject
        implements Adder {
AdderRemote() throws RemoteException {
  super();
}
Public int add(int x, int y)
{
  return x+y;
}
}
```

(iii) Server class :-

```
import Java.rmi.*;
import Java.rmi.registry.*;
Public class MyServer {
public static void main(String args[]) {
try {
  Adder stub = new AdderRemote();
```

Subject :

```
Naming.
}
catch (
System.
}
}
}
```

(iv) Client

Q3. Wh

Subject :

22BCON1068
Ankit Kumar

Date : / /

```
Naming.rebind("rmi://localhost:5000", stub);
}
catch (Exception e) {
System.out.println(e);
}
}
}
```

(iv) Client class:

```
import java.rmi.*;
public class MyClient {
Public static void (String args[]) {
try {
    Adder stub = (Adder)Naming.lookup("rmi://localhost:5000");
System.out.println(stub.add(34,4));
} catch (Exception e)
{
    }
}
}
```

O/P:

Tez 1	Tez 2
(i) Java *.java	Java MyServer
(ii) rmic AdderRemote	
(iii) rmiregistry 5000	

Tez 3

Java MyClient

O:P → 38

Q3. What are the reasons of getting UnknownHostException error even if all the setting are properly configured?

22BCON1068
Ankit Kumar
Date : __/__/__
Subject :

Ans An UnknownHostException in Java typically indicates that the host name could not be resolved to an IP address, even when all setting seem properly configured. Several reasons might still cause this error :-

(i) Incorrect DNS configuration : If the DNS setting on the client or server are misconfigured, the system may not be able to resolve hostnames correctly.

(ii) Network Connectivity Issues :-
- Firewall/proxy Blocking :- Firewalls, proxy servers, or security setting might be blocking communication with the DNS server or the target host, causing hostname resolution to fail.

(iii) Host Unreachable :- The client may not be able to reach the host due to network segmentation, routing issues, or blocked ports. Also the hostname we are trying to reach might not be registered in the DNS system or the hostname might have expired from the DNS database.

(iv) Incorrect Hostname - A small typographical error in the hostname being used in the client code can result in a failure to resolve the host.

(v) Proxy misconfiguration :- if a proxy server is used, but its configuration is incorrect, it might block DNS queries or resolve hostnames incorrectly.

(vi) IPV6 vs IPV4 misconfiguration :- misconfigured preference for IPV6 when only IPV4 is supported.

The end

Oct 14, 2024, 18:03